Optimizing the Robotic Sled Dog's Locomotion Using Reinforcement Learning

Weiching Chen

Diyu Zhou

Kevin Shuhan Liang

Optimization Problem

Optimizing the Robotic Sled Dog's Locomotion

The robotic sled dog should be able to follow forward movement commands with stable and physically plausible postures—without unnecessary spinning in place—while navigating around obstacles and adapting to various terrains and environments.

Objective:

1. Ensure the robot moves with a stable, non-spinning, and physically reasonable gait.
2. Enable the robot to effectively avoid obstacles.

Variables:

1. Different environments and terrain types.
2. Variations in forward movement commands (e.g., distance, speed).

Testing Robot - Unitree Go2

Stability Test

Our Plan:

✅ **Set up environment for simulation – done**

✅ **Revise code, ensure functionality, and run – done**

✅ **Train and ensure a single Unitree Go2 robot dog walks well – done**

✅ **Field trip XD – done**

✅ **Design model (dog pulling sled) – done**

✅ **Convert model to URDF and USD files – done**

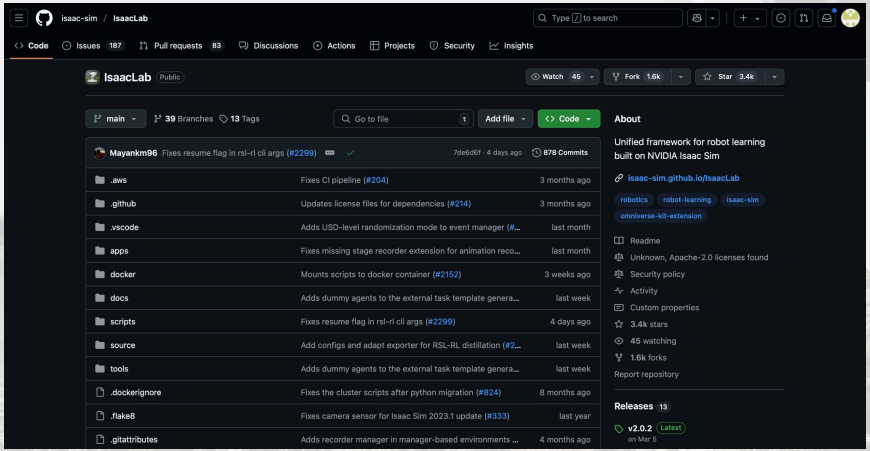
✅ **Import model, train, and evaluate results – done**

🔧 **Adjust constraints and tweak values for better performance – *in progress***

🙏 **Hope it works well >w<"**

Step 1 : Download Isaac Lab to run the simulation.

Requirement for computer:
Ubuntu 22.04
RTX 4060



Step 2 : Set a reward and constraint if the robot meets the goal.

We established a comprehensive set of constraints across three main categories—**robot body constraints**, **joint constraints**, and **foot constraints**—to ensure the robot maintains a physically valid and stable posture. These include:

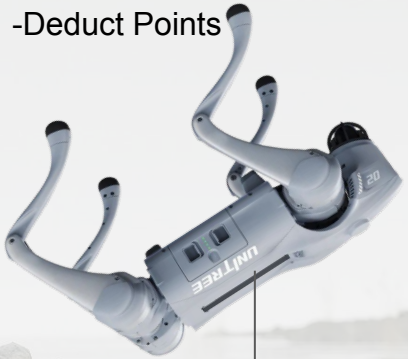
1,Robot-level constraints:
Limitations on overall velocity
Constraints on base height (distance from the ground), etc.

2,Joint-level constraints:
-Restrictions on joint velocity, torque, and power output

3,Foot-level constraints:
-Control of foot height
-Avoiding foot-ground contact when unintended
-Preventing foot sliding or stumbling
-Managing time spent in the air, etc

+Bonus Points

-Deduct Points



Set height, cannot too low
When height >= 38cm, means standing, give reward

Step 2 : Set a reward and constraint if the robot meets the goal.

We established a comprehensive set of constraints across three main categories—**robot body constraints**, **joint constraints**, and **foot constraints**—to ensure the robot maintains a physically valid and stable posture. These include:

1,Robot-level constraints:
Limitations on overall velocity
Constraints on base height (distance from the ground), etc.

2,Joint-level constraints:
-Restrictions on joint velocity, torque, and power output

3,Foot-level constraints:
-Control of foot height
-Avoiding foot-ground contact when unintended
-Preventing foot sliding or stumbling
-Managing time spent in the air, etcc

+Bonus Points

The joint maintains a normal and stable angle.

-Deduct Points

The joint angle is approaching its boundary limit, resulting in postural instability.

Step 2 : Set a reward and constraint if the robot meets the goal.

We established a comprehensive set of constraints across three main categories—**robot body constraints**, **joint constraints**, and **foot constraints**—to ensure the robot maintains a physically valid and stable posture. These include:

1,Robot-level constraints:
Limitations on overall velocity
Constraints on base height (distance from the ground), etc.

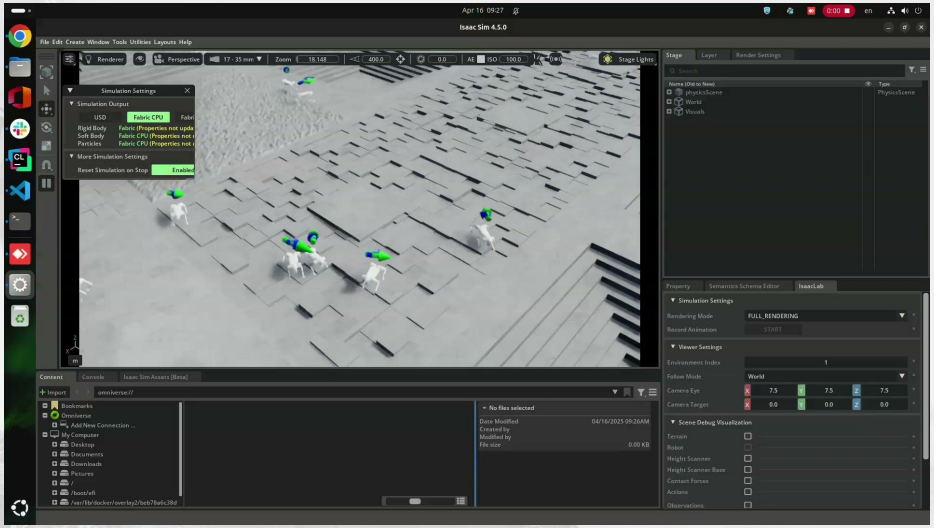
2,Joint-level constraints:
-Restrictions on joint velocity, torque, and power output

3,Foot-level constraints:
-Control of foot height
-Avoiding foot-ground contact when unintended
-Preventing foot sliding or stumbling
-Managing time spent in the air, etc

-Deduct Points

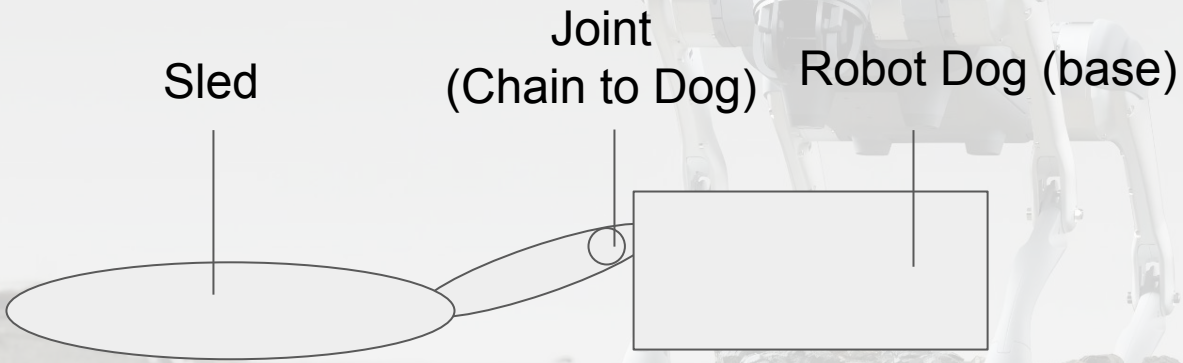
sliding

Step 3 : First run and Result



Step 4 : Design Model

Step 5 : Write URDF




```
<link
  name="base">
  <inertial>
    <origin
      xyz="0.021112 0 -0.005366"
      rpy="0 0 0" />
    <mass
      value="6.921" />
    <inertia
      ixx="0.02448"
      ixy="0.00012166"
      ixz="0.0014849"
      iyy="0.098077"
      iyz="-3.12E-05"
      izz="0.107" />
  </inertial>
  <visual>
    <origin
      xyz="0 0 0"
      rpy="0 0 0" />
    <geometry>
      <mesh
        filename="package://go2_description/dae/base.dae" />
    </geometry>
    <material
      name="">
      <color
        rgba="1 1 1 1" />
    </material>
  </visual>

  <visual>
    <origin xyz="0 0 0" rpy="1.57 0 0"/>
    <geometry>
      <mesh filename="package://go2_description/dae/sled/mesh1.obj" scale="1 1 1"/>
    </geometry>
  </visual>

  <collision>
    <origin rpy="0 0 0" xyz="0 0 0" />
    <geometry>
      <box size="0.3762 0.0935 0.114" />
    </geometry>
  </collision>
</link>
<!-- Sled Link -->
<link name="sled">
  <visual>
    <origin xyz="0.4 0 -0.00078" rpy="1.57 0 0"/>
    <geometry>
      <mesh filename="package://go2_description/dae/sled/mesh0.obj" scale="1 1 1"/>
    </geometry>
  </visual>
  <collision>
    <origin xyz="0.4 0 -0.00078" rpy="1.57 0 0"/>
    <geometry>
      <mesh filename="package://go2_description/dae/sled/mesh0.obj" scale="1 1 1"/>
    </geometry>
  </collision>
  <inertial>
    <origin xyz="-1.4 0 -0.00078" rpy="1.57 0 0"/>
    <mass value="0.5"/>
    <inertia
      ixx="0.03"
      ixy="0.0"
      ixz="0.0"
      iyy="0.0966667"
      iyz="0.0"
      izz="0.0966667"/>
  </inertial>
</link>

<!-- Joint from chain to sled -->
<joint name="chain_to_sled" type="revolute">
  <origin xyz="-0.4 0 0" rpy="0 0 0" />
  <parent link="base"/>
  <child link="sled"/>
  <axis xyz="0 0 1"/>
  <limit effort="5.0" velocity="1.0" lower="0" upper="3.14"/>
</joint>
```

Step 5 : Write URDF

Link: You need to add the 3D model and physical properties such as collision and inertia matrix.

Joint: The joint connecting the sled and the chain can rotate. Set the type to "revolute". To make training easier, we limited the rotation to only the Z-axis, with a range of 0 to 180 degrees.

Moment of Inertia Matrix

Angular Momentum:

$$\vec{L} = \sum m_\alpha \vec{r}_\alpha \times \vec{v}_\alpha = \sum m_\alpha \vec{r}_\alpha \times (\vec{\omega} \times \vec{r}_\alpha)$$
$$= \sum m_\alpha |\vec{r}_\alpha|^2 \vec{\omega} - m_\alpha \vec{r}_\alpha (\vec{\omega} \cdot \vec{r}_\alpha)$$

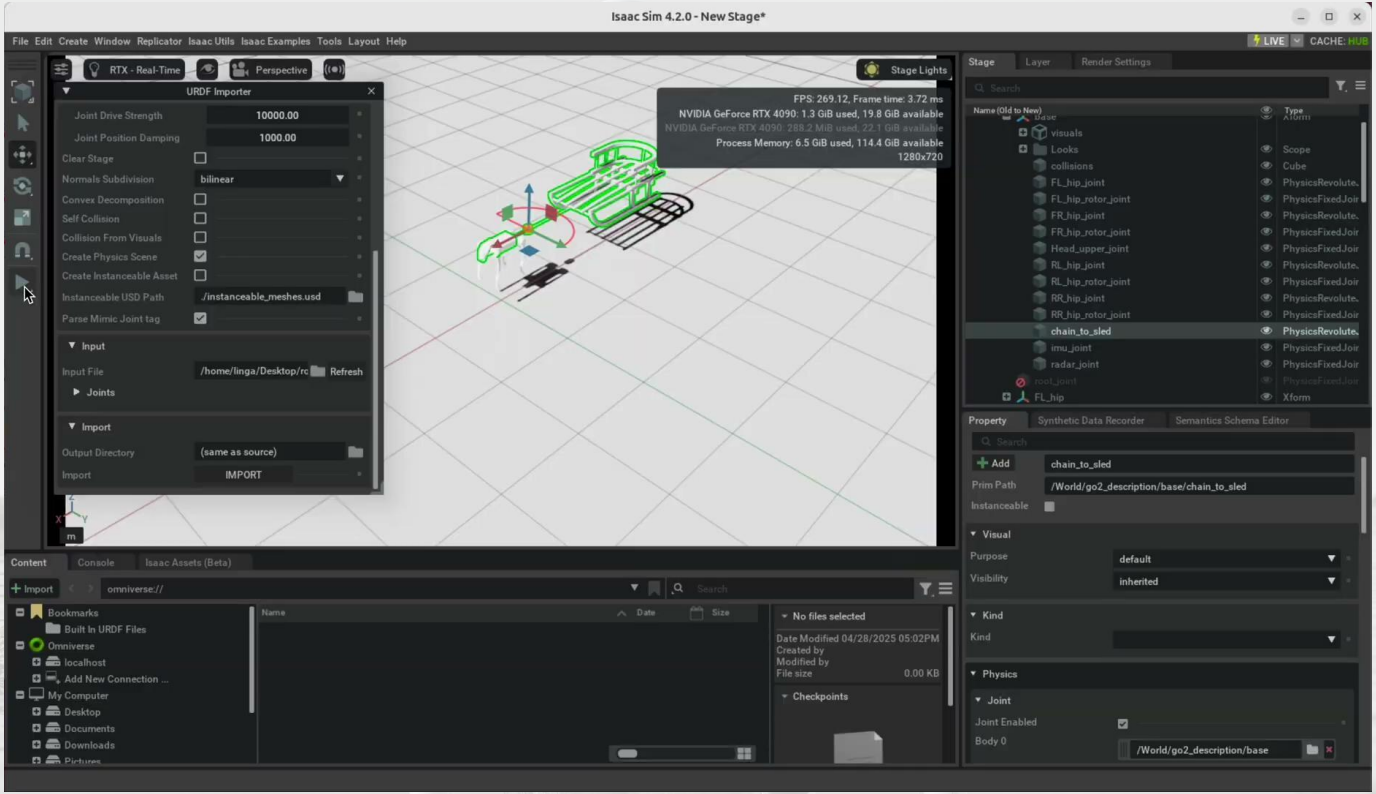
Matrix Equation:

$$L_i = \sum_j \left(\sum m_\alpha r_\alpha{}^2 \delta_{ij} - m_\alpha r_{\alpha i} r_{\alpha j} \right) \omega_j$$

$$I_{ij} = \left(\sum m_\alpha r_\alpha{}^2 \delta_{ij} - m_\alpha r_{\alpha i} r_{\alpha j} \right)$$

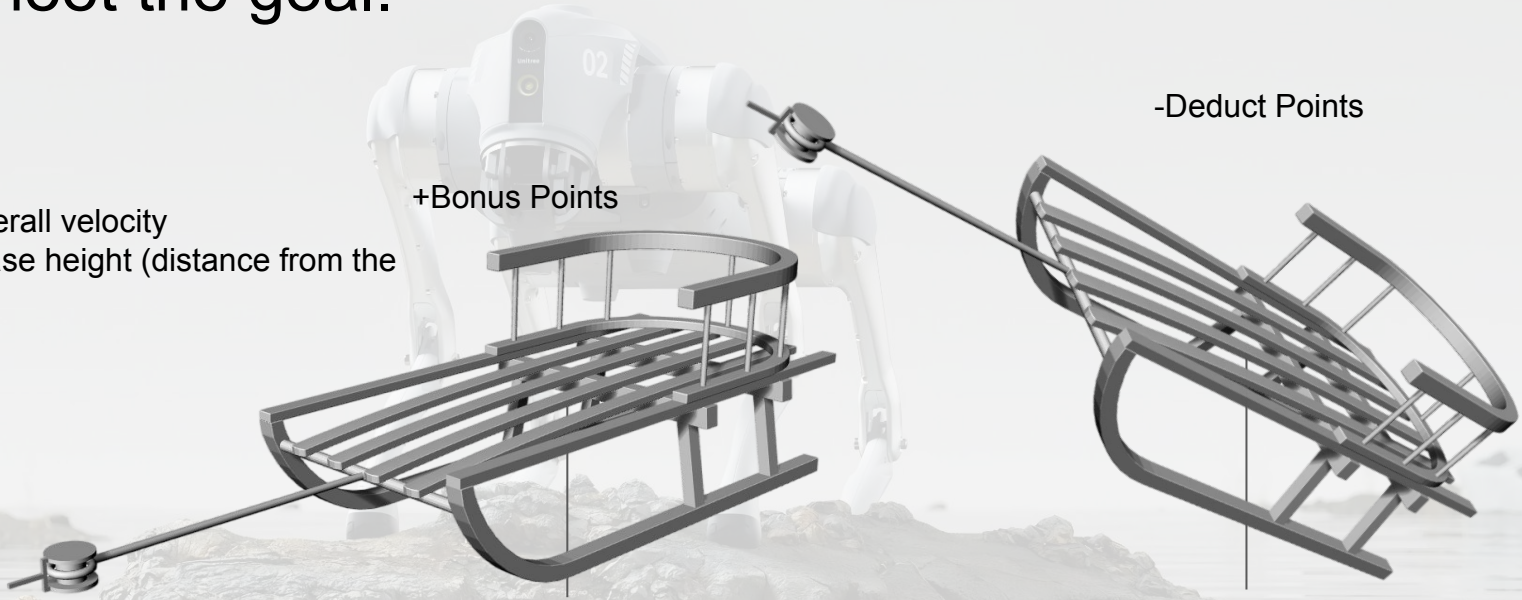
$$[L] = [I][\omega]$$

Step 6 : Test it in Isaac Sim



Step 7 : Set extra reward and constraint if the robot and sled to meet the goal.

Sled constraints:
Limitations on overall velocity
Constraints on base height (distance from the ground), etc.



Set height, cannot too low When height is closed to 30cm, means standing, give reward

Step 7 : Set extra reward and constraint if the robot and sled to meet the goal.

-Deduct Points

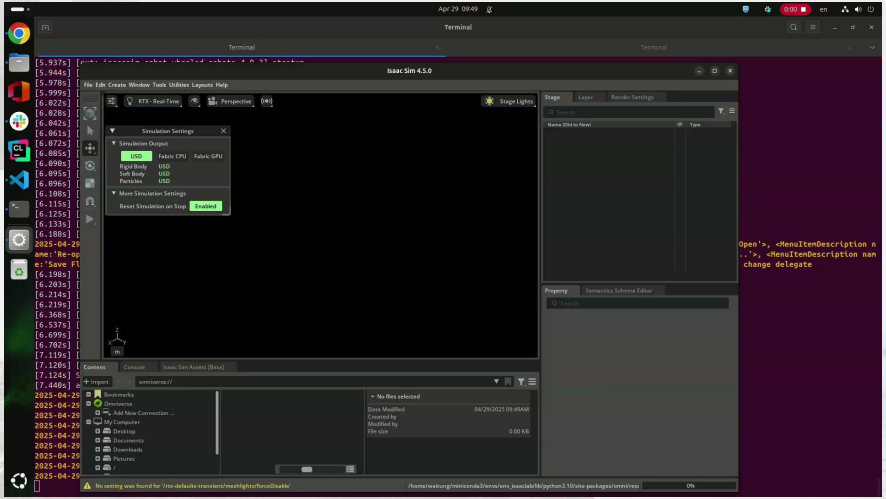
Angle of direction constraints:
Constraints on the angle in between dog facing direction and input direction.

+Bonus Points

Angle = 0

Angle = 30
(turn left)

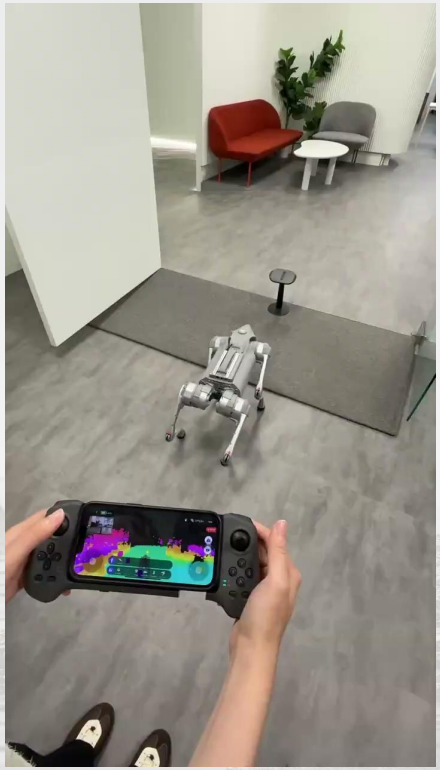
Result

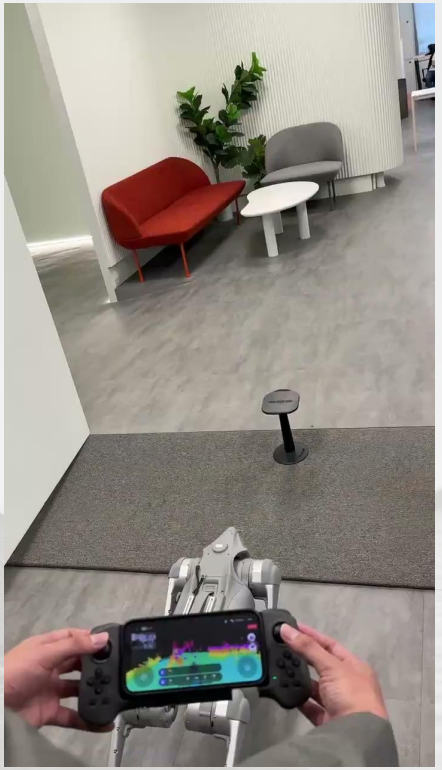


Apply on real robot




Avoid to get into barrier and climb

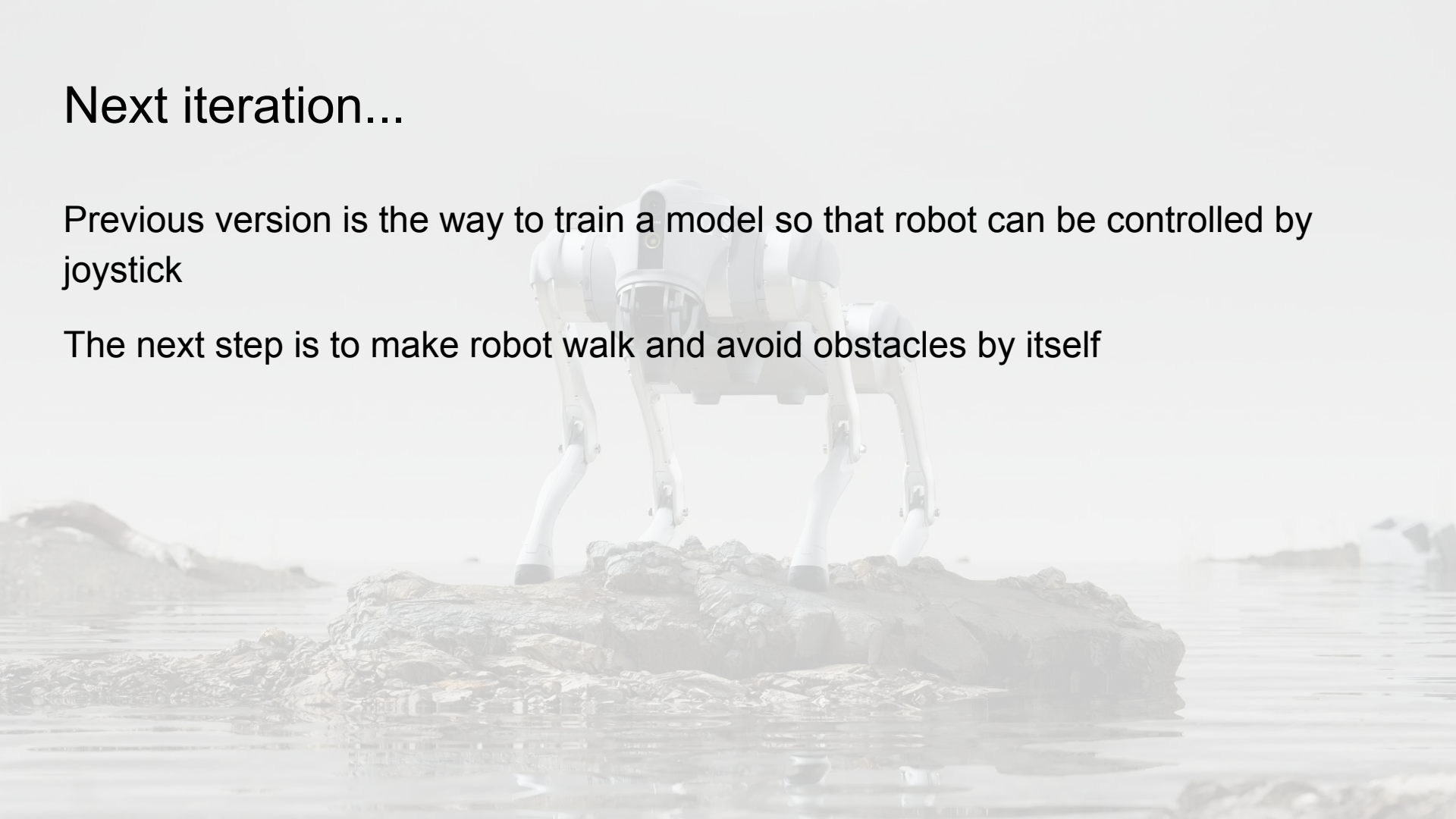




Next iteration...

Previous version is the way to train a model so that robot can be controlled by joystick

The next step is to make robot walk and avoid obstacles by itself



Step 7 : Set extra reward and constraint if the robot and sled to meet the goal.

Robot constraints:
Constraints on the distance of itself and barriers.

-Deduct Points

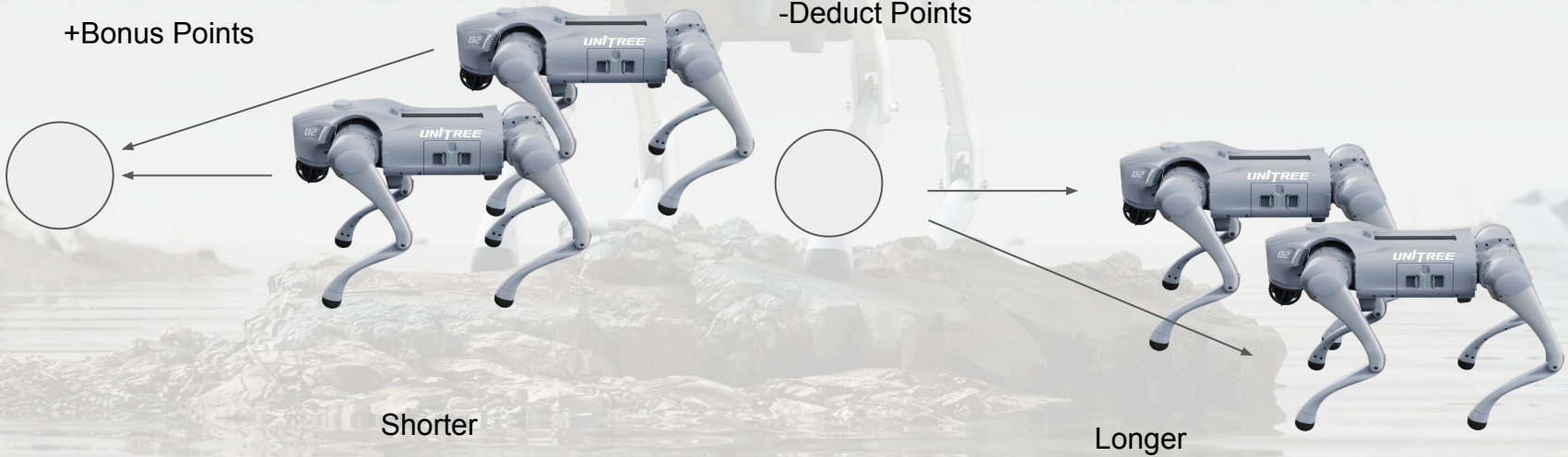
Shorter than last step

+Bonus Points

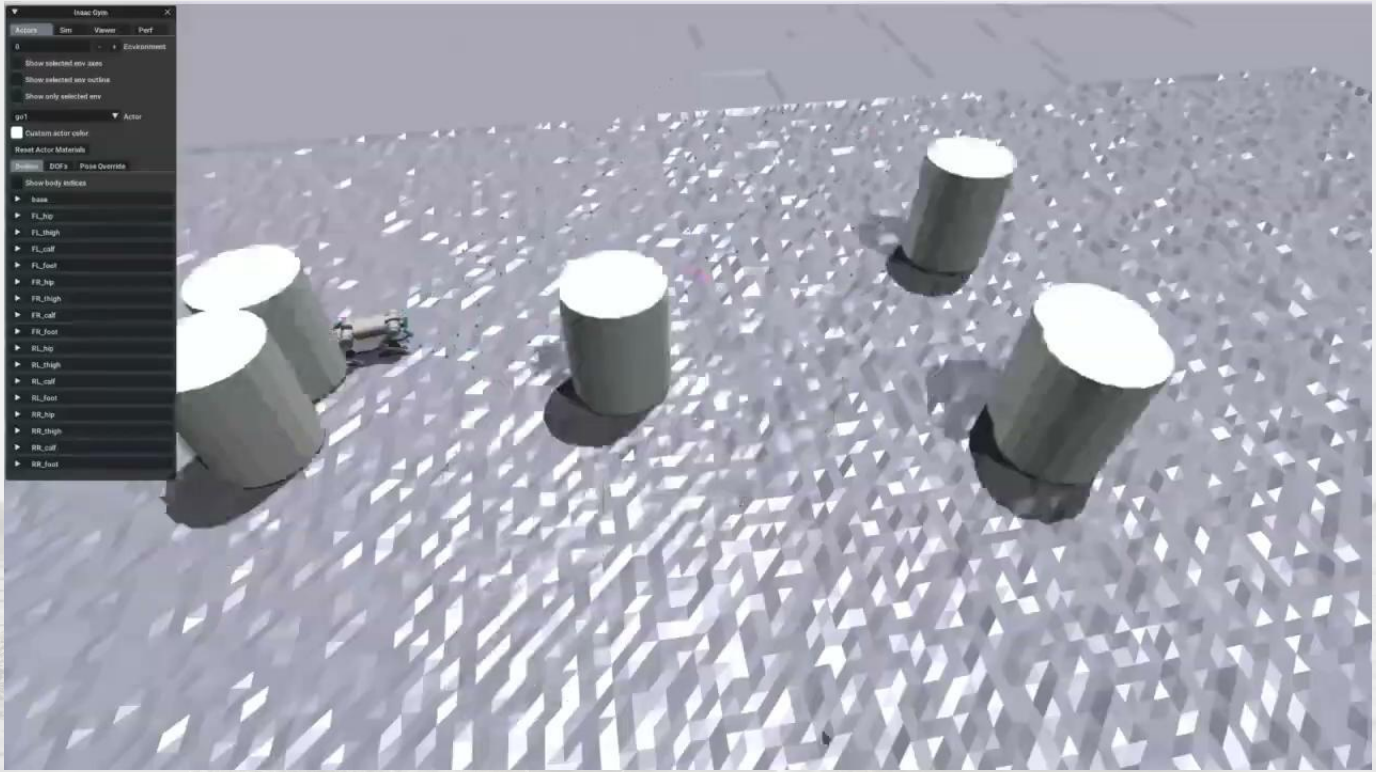
Longer than last step

Step 7 : Set extra reward and constraint if the robot and sled to meet the goal.

Robot constraints:
Constraints on the distance of itself and final goal.

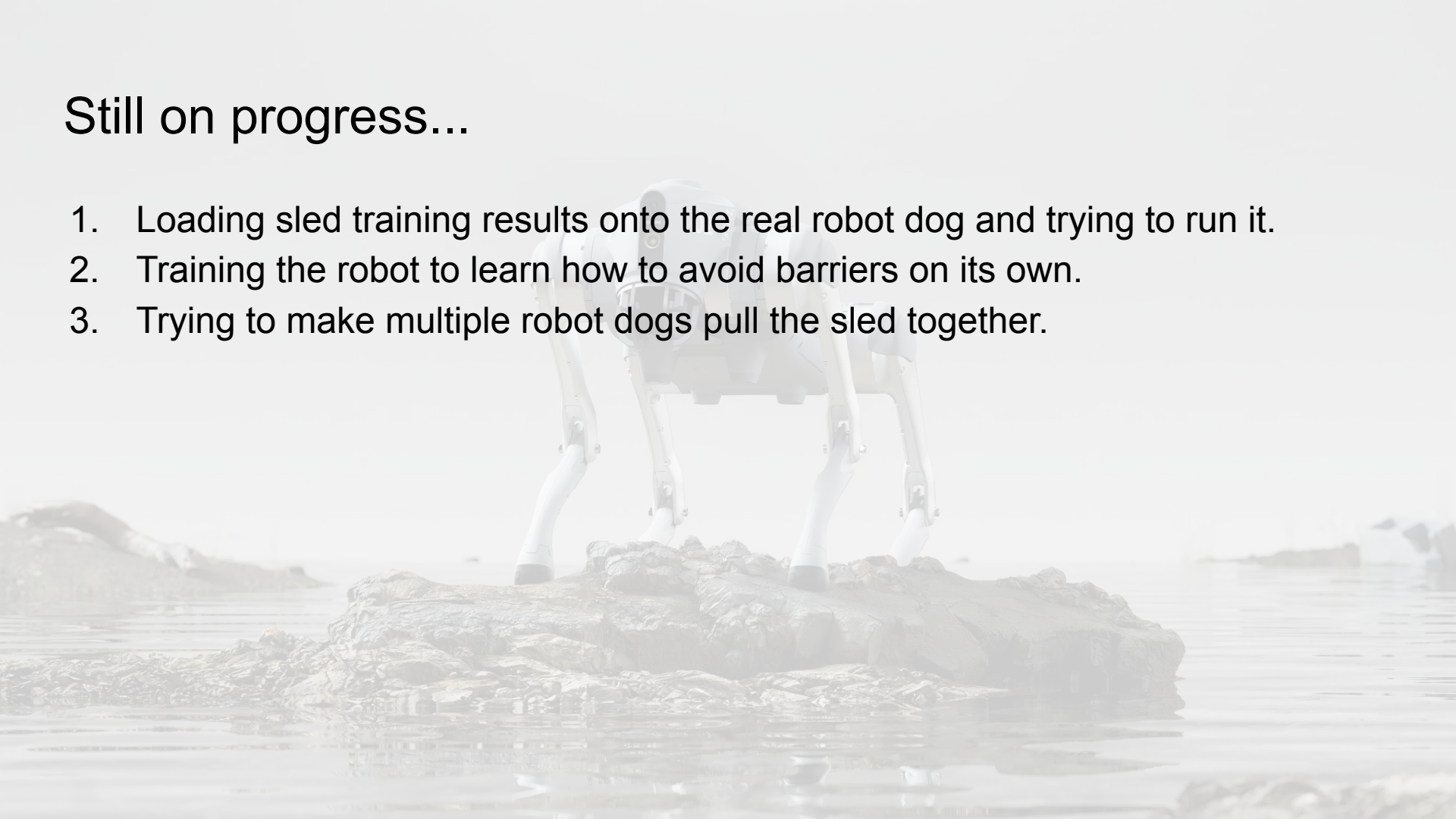


Result


Isaac Gym
Accurs
Sim
Viewer
Perf
Environment
Show selected env axes
Show selected env outline
Show only selected env
go1
Actor
Custom actor color
Reset Actor Materials
Rotation
DOFs
Pose Override
Show body indices
base
FL_hip
FL_thigh
FL_calf
FL_foot
FR_hip
FR_thigh
FR_calf
FR_foot
RL_hip
RL_thigh
RL_calf
RL_foot
RR_hip
RR_thigh
RR_calf
RR_foot

Still on progress...

1. Loading sled training results onto the real robot dog and trying to run it.
2. Training the robot to learn how to avoid barriers on its own.
3. Trying to make multiple robot dogs pull the sled together.




Thanks!